

How merging works in the script

The HTML page likely reads each ODT file as a ZIP and parses its `content.xml`, then iterates through the document's paragraph nodes (e.g. `<text:p>` elements) and appends them into a single HTML container. In other words, it probably does something like:

```
// Pseudocode for merging ODT content
let mergedContainer = document.createElement('div');
for (let file of [doc1, doc2]) {
  let xml = unzipAndParseXML(file);           // open ODT and get
content.xml
  let paras = xml.getElementsByTagName('text:p');
  for (let p of paras) {
    // Create a corresponding HTML <p> and append to mergedContainer
    let htmlP = document.createElement('p');
    htmlP.textContent = p.textContent;         // copy paragraph text
    // (copy any needed styles here)
    mergedContainer.appendChild(htmlP);
  }
}
document.body.appendChild(mergedContainer);
```

This matches common ODT-merge logic: import each paragraph node from the first doc, then from the second, in sequence. (If using pure HTML/DOM methods like `importNode` or cloning nodes, the effect is similar.)

Why large spacing causes overlap

When the **first** document contains big vertical space (e.g. blank lines or large “space-after” on a paragraph), that space must be preserved in HTML. In ODT, an empty paragraph (`<text:p style-name="...">`) represents a blank line ¹. If the script *ignores or skips* those empty `<text:p>` tags (for example, by doing `if (p.textContent.trim() === "" ) continue`), then the blank line vanishes and the merged content from the second doc moves up into that gap. In effect, the second document's content appears “inserted into” the whitespace of the first.

Moreover, even if blank `<p>`s are preserved, CSS can cause unintended collapsing: by default, vertical margins of adjacent blocks collapse into one. For example, if the last paragraph of Doc1 has a 50px bottom margin and the first paragraph of Doc2 has a 30px top margin, the **effective** gap might not be 80px but only 50px (the larger one) ². In extreme cases of empty blocks, margins may even collapse completely ³. This CSS behavior can make large spaces disappear or overlap unexpectedly. In short: **if blank**

paragraphs are lost or margins collapse, the second doc's text will overlap the empty region of the first.

Fix: preserve the blank space in HTML

1. Preserve empty paragraphs. In the merge loop, do *not* skip empty `<text:p>`. Instead, explicitly output a blank HTML paragraph (e.g. `<p> </p>`) for each empty ODT paragraph. For example, in the script's loop:

```
for (let p of firstDoc.getElementsByTagName('text:p')) {
  let htmlP = document.createElement('p');
  let txt = p.textContent.trim();
  if (txt === "") {
    // Insert an explicit blank line so space is not lost
    htmlP.innerHTML = "&nbsp;";
  } else {
    htmlP.textContent = txt;
  }
  mergedContainer.appendChild(htmlP);
}
// ... repeat for second document
```

This way, every blank line in the .odt is kept as an actual paragraph in the merged HTML, preserving vertical space ¹. (If the code was using `innerHTML += ...` or text cloning, switch to using `appendChild` or `importNode` so empty nodes aren't dropped.)

2. Prevent margin collapse. If margins are still causing overlap, wrap each document's content in its own container or give the merged container a new block context. For example, add a CSS rule like:

```
.merged-section { overflow: auto; /* or display: flow-root; */ }
```

and wrap each doc's HTML in `<div class="merged-section"> ... </div>`. This ensures the vertical margins of one section do not collapse with the next ³. As MDN notes, margins of adjacent blocks collapse unless there is padding/border or a new block formatting context ³. By giving the container e.g. `display: flow-root` or a small transparent padding, you force the blank space to be honored.

3. (Alternate) Insert a visible separator. As a simple workaround, after merging Doc1 you could insert a full-width invisible separator (like `<hr style="visibility:hidden;clear:both;">`) or set the container's CSS (e.g. `padding-bottom:1px;`) before appending Doc2. This break would hold open the spacing.

In summary, ensure the code *includes* each blank paragraph (don't trim it away) and/or adjust the CSS to prevent collapsing. For example:

```
// After appending first doc content:
mergedContainer.appendChild(document.createElement('div')).classList.add('merged-
section');
// Then append second doc content inside that div.
```

This ensures large “paragraph spacing” in the first ODT turns into an actual gap in the merged HTML, so Doc2’s text won’t slide up into it.

Answer summary: The script merges by copying all paragraphs from Doc1 then Doc2 into one DOM. The overlap happens because empty paragraphs (blank lines) or large margins in the ODT aren’t preserved properly. By treating empty `<text:p>` as real paragraphs (`<p> </p>`) and/or by preventing CSS margin collapse (e.g. using a new block container or padding), you keep the vertical white space. That change will make the .odt merge add content *below* the gap instead of on top of it.

Sources: In ODT, an empty `<text:p>` is literally “a blank paragraph (an empty line)” ¹. In HTML/CSS, adjacent vertical margins may collapse (combining to the larger gap) ² ³, so explicitly inserting or preserving empty elements prevents that collapse and preserves spacing.

¹ OpenDocument technical specification - Wikipedia

https://en.wikipedia.org/wiki/OpenDocument_technical_specification

² ³ Mastering margin collapsing - CSS | MDN

https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_box_model/Mastering_margin_collapsing